Fast-Weight Memory Layers Improve Small Language Models at Matched Parameters and Compute

Dimitar Stojanovski
Aware AI Labs dimitar@awareailabs.com

July 2026

Abstract

We replace every second attention layer of a modern Transformer with a gated delta-rule fast-weight memory layer and evaluate the result under strict accounting: parameters matched to 0.1%, identical data, matched training budget, and pre-registered decision margins. On five procedural task families, the memory hybrid is more accurate on all five (+4.0 to +21.8 points) while spending 20–40% *fewer* inference FLOPs per correct answer (Figure 1). A 2×2 ablation attributes both effects to the delta mechanism (+22.8) rather than its sliding-window component (+3.5); doubling the baseline's training budget does not close the gap (+25.8 on non-saturated difficulty tiers); and the preferred memory density is every second layer. At 50M parameters under a fixed recipe the advantage is task-dependent: it grows on program execution (+31.7) and reverses on state tracking (−5.2). We additionally observe bimodal skill acquisition: 1/6 seeds transitions discontinuously to 100% accuracy on a rule-revision task (0/6 for the baseline). The complete study — eight pre-registered experiments including two negative results on recurrent-depth loops — cost $38 of commodity cloud compute. Code, per-run results, and pre-registrations are public.[1]

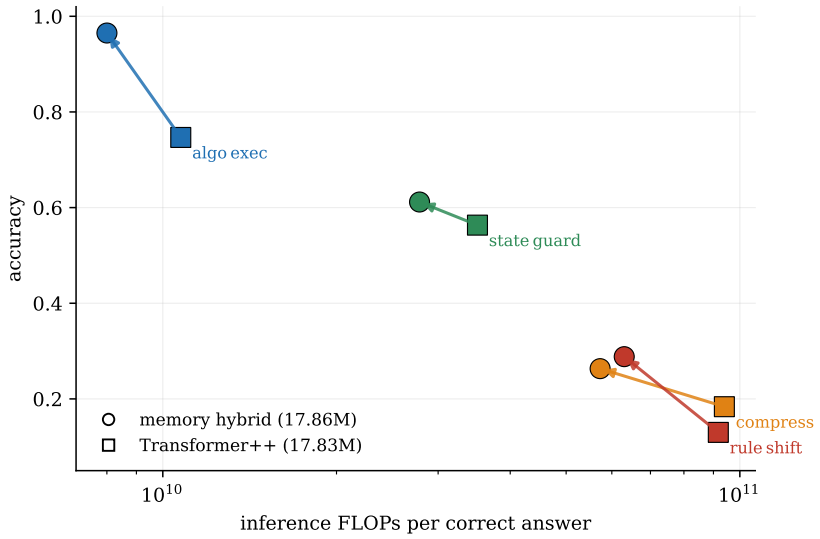

Figure 1: Main result. Accuracy versus inference FLOPs per correct answer at 17.8M parameters, same data, same training budget (means over 3–6 seeds; arrows point from Transformer++ to the memory hybrid). The hybrid is up-left — more accurate *and* cheaper per correct answer — on every family. (`dsl_learn`, +4.0, omitted for legibility.)

[1] https://github.com/dimitri-sky/fast-weight-memory-lm (paper) and https://github.com/dimitri-sky/aware-latent-depth (full research program, run-level ledger).

1 Introduction

Linear-attention hybrids with delta-rule fast weights are entering production language models, yet public evidence for *why* they win is thin: papers rarely hold parameters, data, and compute fixed simultaneously, and per-answer inference cost is almost never reported. We contribute a small, fully audited data point. The question is narrow and the accounting is strict; every claim below was given a pre-registered pass/fail margin before results existed, and every number traces to a run identifier in a public ledger.

Concretely, we compare a byte-level Transformer++ baseline (RoPE, SwiGLU, RMSNorm, GQA; 17.83M) against a hybrid that replaces every second attention layer with a *gated delta-rule fast-weight layer* — a linear-attention state $S_t \in \mathbb{R}^{d_k \times d_v}$ updated token-by-token with a learned write/erase gate — keeping sliding-window attention (w=128) on the remaining layers (17.86M, +0.1%). Both are trained from scratch, per-family, on SAGE: five generated, contamination-audited procedural families (program execution, mid-session rule revision, in-session codebook learning, state tracking, fresh-language induction) with difficulty tiers and execution-checked scoring.

2 Setup

Accounting. FLOPs are counted analytically (2 FLOPs per MAC) and cross-checked against PyTorch's counter in CI (agreement within 10% enforced). Every generated token is charged a full forward pass at its context length. The efficiency metric is *inference FLOPs per correct answer*, which charges mistakes to the model that makes them.

Protocol. 4,000 steps × batch 32 × 768 tokens per family; learning rate depth-scaled; 3 seeds (6 where noted, 2 for trend reads, all stated in advance). Margins, gates, and deviations are logged in time-stamped pre-registration files; the two mid-study deviations (an arm dropped for cost before its results existed; a config bug caught by a feasibility probe) are recorded there.

3 Results

3.1 The memory hybrid wins on all five families, at lower cost

Table 1: 18M head-to-head. Mean accuracy ± SD over seeds; Welch t reported per family. FLOPs/correct is inference cost per correct answer.

family	Transformer++	memory hybrid	Δ	Welch t	FLOPs/correct ratio
algo_exec	.747 ± .016	**.965 ± .009**	+21.8	20.7	0.75×
rule_shift[6s]	.130 ± .034	**.288 ± .349**	+15.8	1.1	0.69×
compress	.184 ± .009	**.263 ± .032**	+7.9	4.1	0.61×
state_guard	.563 ± .043	**.612 ± .028**	+4.8	1.7	0.79×
dsl_learn[2s]	.165 ± .014	**.205 ± .021**	+4.0	2.2	—

The pooled family-mean gap is +13.5 points against a pre-registered +3.0 margin. Statistical honesty: the gap is individually significant on the two largest effects (algo_exec t=20.7, compress t=4.1) and directional but not individually significant on the rest at these seed counts; `rule_shift`'s mean is dominated by one grokked seed (median gap +2.5). The pre-registered 2×pooled-SD stability gate fails for exactly this reason, and we report it as such.

3.2 Attribution: the delta mechanism, not the window

The hybrid differs from the baseline in two ways, so we complete the 2×2 (Figure 3). Adding the sliding window alone moves the baseline +3.5; adding delta layers alone moves it +22.8 (.975, slightly *above* the full hybrid); the interaction is −4.5. The window also fails to reproduce any memory-family gain (state

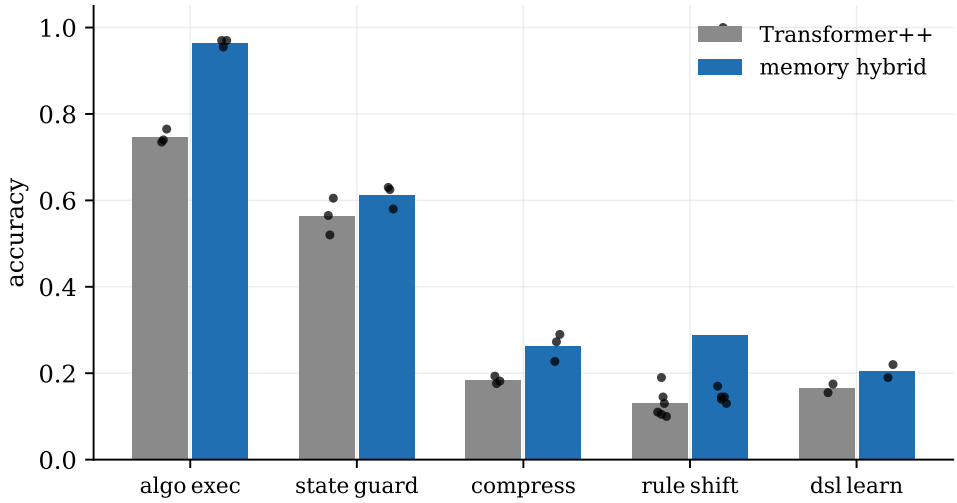

Figure 2: Per-family accuracy with individual seeds (dots). The hybrid leads on 5/5 families; `rule_shift`'s variance is the bimodality of Figure 6.

tracking *degrades* by 4.1). A pre-registered prediction was falsified here: we expected the window to explain the FLOP discount, but the discount is also delta-driven — the window trims only ∼7% of FLOPs; the rest comes through the accuracy denominator.

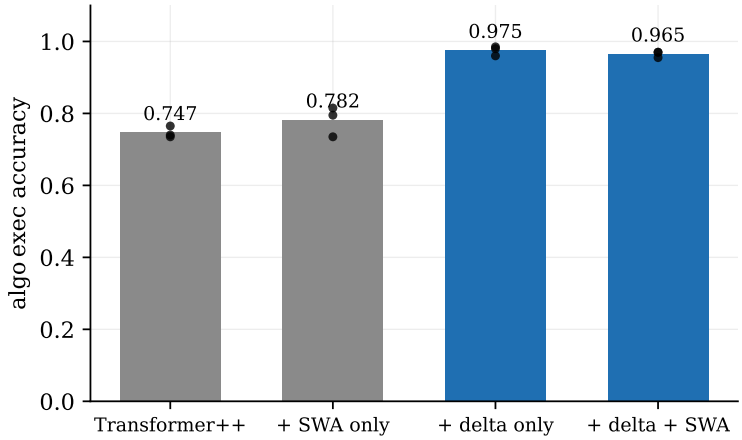

Figure 3: 2×2 ablation on `algo_exec` (3 seeds each, dots). The delta mechanism carries the effect.

3.3 Doubling the baseline's training budget does not close the gap

If delta layers merely *optimize faster*, more training should let the baseline catch up. At 2× steps the baseline improves (.747→.795 — the arm discriminates) but the gap on non-saturated tiers (3–5) remains +25.8 points against a pre-registered pass bar of half the original gap (Figure 4). A ceiling guard (hybrid $> .95$ overall) moved the primary read to hard tiers, as pre-registered.

3.4 At 50M the advantage is task-dependent

Scaling both models to 50.5M (−0.05% param match) under the same recipe: the program-execution gap *grows* to +31.7 while the state-tracking gap *reverses* to −5.2 (Figure 5). A caveat cuts both ways: the 50M

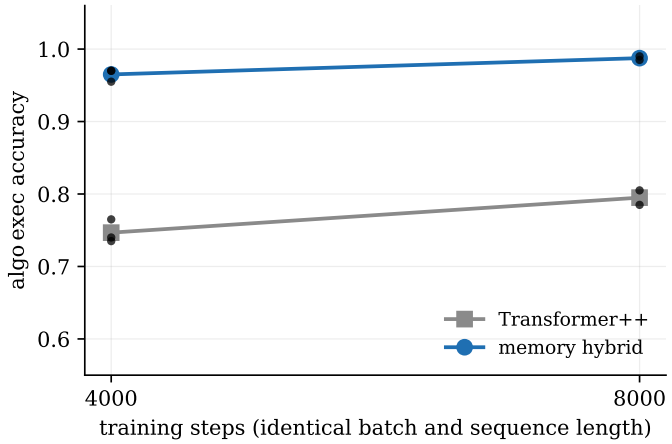

Figure 4: Training-budget robustness on `algo_exec`.

baseline underperforms its own 18M counterpart on `algo_exec`, indicating the fixed recipe undertrains at this size — part of the growth is baseline weakness, and the reversal may be a tuning artifact. We therefore make no uniform-scaling claim.

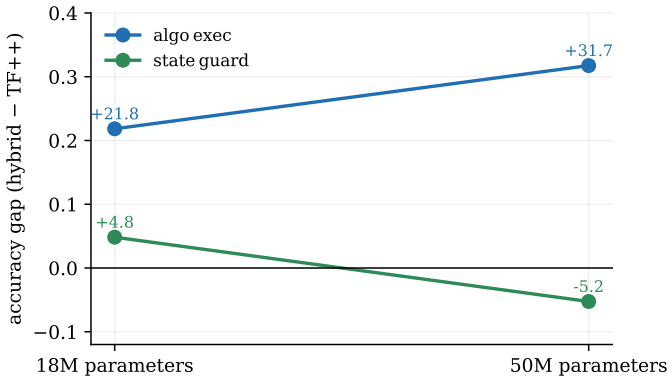

Figure 5: Accuracy gap at 18M vs. 50M (fixed training recipe, 2 seeds at 50M).

3.5 Bimodal skill acquisition

On `rule_shift`, five of six hybrid seeds land at .13–.17 and one reaches 1.00 (train loss 0.0014) — a discontinuous transition absent in all six baseline seeds (Figure 6). Same data, same configuration, different initialization. Checkpoints are retained for a follow-up study on early-training predictors of the transition.

3.6 Negative control: recurrent depth does not pay here

For completeness: in the same program, weight-tied loop architectures (prelude–core$\times K$–coda) failed to pay their FLOP cost twice — vanilla loops lose to a step-matched control, and the current best training recipe (per-loop supervision, randomized loop counts, truncated BPTT) twice produced *loop-invariant* models, detected only by a pre-registered diagnostic that evaluates the same checkpoint at $K{=}1$ vs. $K{=}4$ ($<$ 2-point gap against a 5-point gate). Latent computation helped here only when it carried *writable state across the sequence*, not when it re-applied the same transformation in depth.

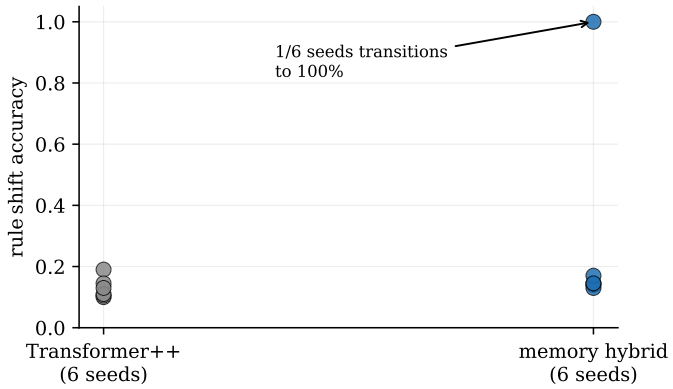

Figure 6: Seed-level outcomes on `rule_shift` (6 seeds per model).

4 Limitations

15–50M scale; procedural byte-level benchmark; per-family specialists (no multi-task control); one window size; 2–6 seeds; the 50M recipe untuned. The delta scan is implemented sequentially, so wall-clock favors the baseline (FLOP counts are implementation-independent; chunked-parallel kernels for this layer family exist in production systems). Small-scale results do not license frontier claims; the mechanism family is already validated at scale by others — what this study adds is the controlled, matched, pre-registered evidence chain and the per-answer cost accounting.

5 Reproducibility

All pre-registrations (`agent/log/EXP-*.md`), the 1,300-row run ledger (`experiments/results.csv`), frozen configs, adjudication scripts, and a two-minute demo (`scripts/demo.py`: fresh puzzles, both models side-by-side, live FLOPs-per-correct meter) are public. Total compute: one local RTX 3080 plus ∼55 RTX 5090 GPU-hours ($38).

References

[1] S. Yang et al. Gated Delta Networks: Improving Mamba2 with Delta Rule. *ICLR*, 2025. arXiv:2412.06464.

[2] I. Schlag, K. Irie, J. Schmidhuber. Linear Transformers are Secretly Fast Weight Programmers. *ICML*, 2021. arXiv:2102.11174.

[3] Kimi Team. Kimi Linear: An Expressive, Efficient Attention Architecture. 2025. arXiv:2510.26692.

[4] J. Geiping et al. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. 2025. arXiv:2502.05171.

[5] Ouro team. Scaling Latent Reasoning via Looped Language Models. 2025. arXiv:2510.25741.

[6] Readout blind spot in per-loop supervised looped models. 2026. arXiv:2606.24898.

[7] LOTUS: Latent Optimization via Step-Aligned Supervision. 2026. arXiv:2606.31779.

[8] Recurrent-Depth VLA: Implicit Test-Time Compute Scaling via Latent Iterative Reasoning. 2026. arXiv:2602.07845.

[9] Sparse-Layers: iso-FLOP analysis of weight-tied depth, 16M–305M. 2026. arXiv:2605.09165.